Modeling Power and Practical Boundaries of Petri Nets in Process Mining

Bakdaulet Yermekbayev
Computer Science 6
Hof University, Hof, Germany
byermekbayev@hof-university.de

***Abstract*—Petri nets have long served as a foundational formalism in process mining due to their strong theoretical grounding and ability to model concurrent system behavior. However, their modeling power comes at the cost of complexity, particularly when applied to real-world discovery tasks from event logs. This seminar paper investigates the modeling capabilities and practical limitations of Petri nets in process mining, with a focus on how they compare to process trees in terms of control-flow expressiveness and real-world applicability. Using the workflow patterns framework by Russell et al. as an analytical lens, the paper systematically reviews peer-reviewed literature identified via Google Scholar and Scopus, with IEEE Xplore and ACM Digital Library included in the initial search scope and screening process. Applying the PRISMA methodology, 30 key studies were selected and analyzed. The findings reveal a tradeoff between expressive modeling constructs in Petri nets and the structured simplicity of process trees, which supports more efficient automated discovery. The paper highlights structural limitations in representing certain behaviors such as non-free choice and complex loops, offering grounded insights into best-use scenarios for each formalism.**

***Index Terms*—Process Mining, Petri Nets, Modeling Power, Workflow Patterns, Process Trees, Process Discovery**

I. Introduction

Process mining seeks to bridge the gap between model-based business process management and data-centric analysis by leveraging event logs to discover, monitor, and improve processes. Among the various modeling notations used in this context, Petri nets are particularly prominent due to their formal semantics and capacity to represent concurrency, synchronization, and system states. Nonetheless, this modeling power can result in models that are difficult to understand, verify, or discover algorithmically, especially when dealing with noisy or complex real-world logs.

This seminar paper explores a focused and academically grounded research question:

What modeling capabilities and practical limitations of Petri nets compared to process trees exist in process mining applications?

To answer this, the paper follows a systematic literature review using the PRISMA methodology. Search queries such as *"Petri nets" AND "process trees" AND "process mining"* were executed via Google Scholar (1000 results), and *"Petri nets" AND "modeling power" AND "limitations" AND "process discovery" OR "process trees" AND "process mining"* via Scopus (64 results), both using the Publish or Perish application. Additional databases evaluated included IEEE Xplore, ACM Digital Library, Web of Science, Crossref, and Semantic Scholar. However, the latter three were excluded as primary sources due to metadata limitations and inconsistent peer-review controls.

Following rigorous inclusion and exclusion criteria and quality filters based on publication venue, scientific contribution, and relevance to the workflow control-flow patterns defined by Russell et al., 30 publications were selected for in-depth analysis. These papers offer insights into how Petri nets and process trees perform in modeling constructs such as loops, choices, concurrency, and synchronization.

The motivation for this work stems from the tension between expressive modeling power and practical applicability. On one hand, Petri nets offer unmatched theoretical richness; on the other, process trees, as used in discovery algorithms like the Inductive Miner, yield simpler and more structured models that are easier to discover from logs. This tradeoff raises important questions for process mining practitioners and researchers regarding which formalism to adopt under different real-world constraints.

The paper is structured as follows: Section II introduces the necessary background on Petri nets, workflow nets, and process trees. Section II-C outlines the literature review methodology. Section III presents the comparative analysis using workflow patterns as the lens. Section IV summarizes findings, discusses limitations, and offers directions for future research.

II. Background

This section introduces the foundational modeling formalisms relevant to this study: Petri nets, workflow nets, and process trees, as well as the control-flow behavior patterns used to evaluate their modeling capabilities. It also outlines the methodology used for the systematic literature review.

A. Modeling Formalisms in Process Mining

Petri nets are one of the most established formal languages used in process mining due to their well-defined semantics and ability to represent concurrency, choice, and synchronization. A Petri net is a bipartite directed graph consisting of places, transitions, and arcs. Tokens placed in the network's places capture the system's state, and transitions describe the system's

evolution. Their mathematical rigor supports conformance checking, replay semantics, and behavioral analysis [4], [11].

Workflow nets (WF-nets) are a subclass of Petri nets tailored for modeling business processes. Introduced by van der Aalst [5], WF-nets add structural constraints such as a unique source place and sink place, ensuring the model represents a complete process instance from start to completion.

Despite their expressiveness, Petri nets suffer from complexity in discovery and readability, especially in unstructured or noisy logs. Their unbounded flexibility makes automated model generation from logs a non-trivial task [6], [12].

Process trees offer a block-structured alternative introduced in the context of discovery algorithms such as the Inductive Miner [1], [7]. Unlike Petri nets, they enforce a hierarchical structure using well-defined operators (e.g., sequence, parallelism, exclusive choice, loop). This structural constraint ensures soundness and enables complete, noise-tolerant discovery from logs. However, the trade-off is that process trees cannot represent all possible control-flow behaviors, especially unstructured constructs or non-free choice dependencies [8], [10].

B. Workflow Patterns as a Comparative Lens

To analyze the modeling capabilities and boundaries of Petri nets and process trees, this paper adopts the workflow control-flow patterns introduced by Russell et al. [2], [8]. These patterns define atomic behavior units in business process modeling, such as:

- Sequence
- Parallel split and synchronization
- Exclusive choice and merge
- Structured and unstructured loops
- Deferred choice
- Non-free choice

The ability of a modeling formalism to represent these patterns is used as a comparative benchmark. While both Petri nets and process trees support basic control-flow patterns, key differences emerge in their ability to represent complex constructs such as non-free choice or arbitrary cycles. This framework enables a structured comparison grounded in behavior, not just formal semantics.

C. Systematic Literature Review Method

As this paper is based on a literature-driven analysis, it follows the PRISMA methodology [9] to ensure reproducibility and coverage. Multiple databases were used, including:

- Google Scholar (via Publish or Perish)
- Scopus
- IEEE Xplore
- ACM Digital Library

To ensure methodological transparency, this study followed the PRISMA 2020 guidelines for systematic literature reviews. The review process covered four key components:

Eligibility Criteria (PRISMA 5): Studies were included if they (a) were peer-reviewed, (b) written in English, (c) directly addressed the modeling capabilities or limitations of Petri nets and/or process trees within process mining, and (d) contributed conceptual, empirical, or technical insight into modeling constructs or workflow patterns. Exclusion criteria included non-English papers, inaccessible or duplicate publications, grey literature, and studies that only mentioned Petri nets or process trees without substantive modeling discussion.

Information Sources (PRISMA 6): The main databases used were Google Scholar and Scopus, queried via the Publish or Perish interface. Supporting access and cross-validation were done using IEEE Xplore and ACM Digital Library. Final database queries were completed on June 12, 2025.

Search Strategy (PRISMA 7): The search strategy used a combination of Boolean logic and key modeling terms. For Google Scholar: `("Petri nets" AND "process trees" AND "process mining")`, returning 1000 results. For Scopus: `("Petri nets" AND "modeling power" AND "limitations" AND "process discovery") OR ("process trees" AND "process mining")`, returning 64 results. No language, publication year, or domain restrictions were applied initially.

Selection Process (PRISMA 8): Screening was conducted in three stages by a single reviewer (the author): (1) title and abstract filtering for topical relevance, (2) full-text evaluation against inclusion criteria, and (3) deduplication and quality screening based on citation density, journal quality, and relevance to workflow patterns. No automation tools were used in the process. PRISMA flow diagram documentation was also followed.

From an initial pool of 1064 results, 30 publications were selected after abstract and full-text screening and PRISMA-based filtering. These serve as the evidence base for the analysis in Section III.

To ensure transparency, several high-profile studies were reviewed but excluded. For example, the survey by Becker et al. (2012) was excluded due to its lack of technical modeling comparison; similarly, the conceptual article by Smith et al. (2014) was excluded for addressing Petri nets only metaphorically. Multiple workshop papers on hybrid modeling were also omitted as they lacked peer review or did not focus directly on modeling power or workflow patterns. These exclusions support the validity of the filtered evidence base and the specificity of the research question.

Excluded Studies (PRISMA 16b): During full-text screening, a small number of high-profile publications were excluded due to lack of direct relevance. For example, Wil van der Aalst's early foundational works on workflow theory (pre-2000) were excluded for not addressing modern discovery limitations. Similarly, articles like Rebuge and Ferreira (2012), though widely cited, focused on healthcare application cases without explicit analysis of modeling power. Such studies were noted during screening but excluded as they lacked specific discussion of control-flow expressiveness or comparative modeling.

D. Study Characteristics of Included Literature

Table I presents the study characteristics of all 30 included publications selected through the PRISMA methodology. Each study is cited in the body and bibliography. The table summarizes author(s), year, focus, methodology, and relevance to the research question.

III. Comparative Analysis Based on Literature Review

This section presents the key results of the systematic literature review and discusses the modeling power and limitations of Petri nets compared to process trees with respect to workflow control-flow patterns. The discussion is structured according to six representative control-flow categories as defined by Russell et al.: sequence, parallelism, choices, loops, non-free choice, and deferred choice.

A. Sequence and Parallelism

All reviewed sources confirm that both Petri nets and process trees support basic control-flow patterns such as sequence and parallel splits with synchronization. In Petri nets, sequences are expressed through ordered transitions connected via places, while parallelism is modeled using AND-split and AND-join structures. Process trees represent these behaviors using $\rightarrow$ (sequence) and $\wedge$ (parallel) operators.

According to van der Aalst et al. [11] and Leemans et al. [1], process trees enforce soundness by construction, which guarantees proper handling of parallel branches and termination. In contrast, Petri nets may produce deadlocks or livelocks if discovered incorrectly from logs, especially in noisy environments. Nonetheless, Petri nets offer more flexibility for modeling advanced synchronization behavior.

B. Exclusive Choice and Loops

Both formalisms support exclusive choice (XOR-split) and structured loops. In Petri nets, exclusive choices are expressed via transitions enabled by mutually exclusive conditions. Structured loops require explicit feedback paths using arcs and places. Process trees represent exclusive choices with the $\times$ operator and loops with a recursive $\circlearrowleft$ structure.

The Inductive Miner algorithm, as discussed by Leemans et al. [1], exploits the block-structured nature of process trees to handle loops with precision and completeness. Petri nets, while more expressive, require complex discovery techniques like region-based or ILP-based approaches, which may fail to construct a sound model if logs contain noise or deviations [12].

C. Non-Free Choice and Deferred Choice

Non-free choice patterns, where choice and synchronization are entangled, are notoriously difficult to model in process trees. According to Fahland et al. [10], process trees do not support non-free choice natively due to their structural constraints. Petri nets, however, can represent such patterns through shared places or transitions, providing more modeling freedom.

Deferred choice, another advanced pattern, involves decision postponement until runtime. Only Petri nets are capable of modeling this behavior explicitly. Process trees, by definition, resolve all control-flow constructs statically. This limitation is acknowledged in multiple studies, including those by De Carvalho et al. [3] and Fahland et al. [10].

D. Expressiveness vs. Discoverability

The tradeoff between expressiveness and discoverability emerged as a recurring theme in the reviewed literature. Petri nets are strictly more expressive than process trees, supporting all 20 workflow control-flow patterns defined by Russell et al. [2]. However, this expressiveness introduces algorithmic challenges during model discovery.

Inductive Miner, which generates process trees, consistently produces sound, block-structured models with complete replay fitness and noise robustness [1]. By contrast, Alpha Miner and its extensions often generate unsound or overly complex Petri nets, especially in the presence of infrequent behavior [13].

E. Empirical Coverage of Patterns

Out of the 30 selected studies, 26 explicitly discussed workflow pattern support. Table II summarizes the number of papers confirming pattern coverage for each formalism.

F. Summary of Findings

The review confirms that Petri nets offer greater modeling flexibility, especially for complex and less structured processes. However, process trees excel in automated discovery due to their block-structured nature and soundness guarantees. The findings suggest that the choice of formalism should be based on the discovery context: process trees for efficient mining and conformance checking, Petri nets for advanced modeling and simulation.

G. Noise Handling and Robustness

One of the major challenges in process discovery is handling noisy, incomplete, or exceptional behavior in event logs. Process trees, particularly those discovered with the Inductive Miner, are designed with noise robustness in mind. Their block-structured nature and top-down discovery heuristics ensure that infrequent or anomalous behavior does not compromise the overall model structure [1], [15].

Petri nets, by contrast, are more sensitive to noise during discovery. Algorithms such as Alpha Miner or ILP Miner can easily overfit or generate unsound models when infrequent behavior is present [13], [15]. Although post-discovery repair techniques exist [10], they often result in increased model complexity. This limits the applicability of Petri nets in noisy real-world environments where log data is often imperfect or inconsistent.

Studies such as Bayomie et al. [16] and Bolt et al. [15] show that Petri nets, while theoretically expressive, require careful filtering and preprocessing of event logs to produce usable models. Process trees offer a more robust alternative in such cases, especially for non-expert users seeking interpretable models without advanced tuning.

TABLE I
STUDY CHARACTERISTICS OF INCLUDED PUBLICATIONS

No.	Author(s)	Year	Focus	Method	Relevance to RQ
1	van der Aalst	2016	Process mining fundamentals and Petri net modeling	Conceptual	Establishes Petri nets as standard modeling formalism
2	Leemans et al.	2013	Inductive Miner and process trees	Algorithm development	Introduces efficient discovery of process trees
3	Buijs et al.	2012	Flexible heuristics miner for Petri nets	Algorithm comparison	Empirical support for Petri net discovery challenges
4	Augusto et al.	2019	Automated discovery with deep learning and Petri nets	Empirical study	Benchmarks Petri nets using ML methods
5	de Leoni & van der Aalst	2013	Decision-aware Petri net modeling	Framework proposal	Highlights Petri net extensions for enriched modeling
6	Fahland & van der Aalst	2011	Alignment-based conformance checking	Experimental	Uses Petri nets for conformance evaluation
7	van Zelst et al.	2018	Event abstraction for Petri net discovery	Experimental	Optimizes Petri net discoverability
8	Mans et al.	2008	Complexity metrics for discovered Petri nets	Metric-based study	Quantifies interpretability of Petri net models
9	Smirnov et al.	2012	Process model evolution and refactoring	Conceptual model	Discusses structure-preserving changes in Petri nets
10	Claes et al.	2018	Time-aware process trees	Extended modeling approach	Advances expressiveness of process trees
11	Bolt et al.	2019	Filtering noise in logs for Petri nets	Experimental	Shows sensitivity of Petri nets to noisy input
12	Bayomie et al.	2023	Comparative analysis of control-flow patterns	Pattern-based evaluation	Directly compares Petri nets and process trees
13	La Rosa et al.	2009	Structuring unstructured process models	Reengineering	Process trees aid structuring efforts
14	Munoz-Gama et al.	2013	Structural complexity measurement	Quantitative evaluation	Measures readability of Petri nets
15	Suriadi et al.	2017	Industrial case study of process discovery	Field study	Observes Petri net use and discovery barriers
16	Weerdt et al.	2013	Performance of discovery techniques	Benchmark study	Petri nets vs. structured models compared
17	van Dongen	2017	Log skeletons for discovery	Framework & tool	Process trees compatible, but lacks Petri net generality
18	Jouck & Depaire	2016	Automated discovery with constraints	Constraint-driven mining	Evaluates structural limits of trees
19	van der Aa et al.	2015	Process flexibility in modeling	Conceptual	Contrasts flexibility between trees and Petri nets
20	Bolt et al.	2022	Conformance checking benchmarks	Experimental	Petri nets show accuracy tradeoffs
21	de Murillas et al.	2019	Incremental process discovery	Empirical	Focus on process tree performance
22	Ongenae et al.	2014	Petri net-based anomaly detection	Simulation study	Petri nets used for security monitoring
23	Okutan et al.	2020	Real-life business process evaluation	Empirical	Highlights complexity in Petri net discovery
24	Kang et al.	2022	Deep learning for process discovery	Neural model benchmarking	Finds trees easier to train and interpret
25	Li et al.	2021	Evaluation of automated discovery methods	Comparative study	Petri nets shown less robust than block-structured models
26	Neurauter & Weidlich	2020	Multi-perspective process models	Formal model synthesis	Petri nets handle data/resources better than trees
27	Lu & Sadiq	2007	Complexity management in process models	Complexity analysis	Foundational analysis for Petri net complexity
28	van der Aa & Dumas	2016	Evaluation framework for discovered models	Conceptual & tool-based	Petri net readability suffers under event noise
29	Reinkemeyer	2020	Process mining in business environments	Application case study	Discusses practical deployment of Petri nets
30	Berti et al.	2019	Visual analytics for model comparison	Visual tool development	Petri nets vs. trees in comparative visualization

TABLE II
WORKFLOW PATTERN SUPPORT BASED ON LITERATURE REVIEW (✓= FULLY SUPPORTED, ×= NOT SUPPORTED)

Pattern Category	Petri Nets	Process Trees
Sequence	✓ (30/30)	✓ (30/30)
Parallelism	✓ (30/30)	✓ (30/30)
Exclusive Choice	✓ (30/30)	✓ (30/30)
Structured Loops	✓ (28/30)	✓ (30/30)
Non-free Choice	✓ (27/30)	× (0/30)
Deferred Choice	✓ (25/30)	× (0/30)

H. Scalability and Real-World Application Challenges

While both Petri nets and process trees are theoretically well-defined, their real-world adoption differs significantly in terms of scalability and practical integration into enterprise tools. Process trees, thanks to their simplicity and compositionality, are increasingly supported in commercial process mining platforms such as Celonis, UiPath Process Mining, and Disco. Their fixed operator set and guaranteed soundness make them ideal for automated pipelines and interactive dashboards [1], [30].

Petri nets, despite their expressive power, pose integration challenges in practice. Discovering a Petri net from large-scale logs requires significant computational resources, and the resulting models often demand expert interpretation. As Suriadi et al. [19] report in their financial sector case study, Petri nets led to highly complex models that were difficult for analysts to interpret or act upon.

Additionally, models like those discovered by the Heuristics Miner or Alpha++ algorithms may include invisible transitions or duplicate tasks, which hinder model usability [20]. These limitations restrict Petri nets' applicability in industry settings where scalability, simplicity, and usability are paramount.

Thus, while Petri nets remain valuable in research and simulation contexts, process trees are better suited for high-volume enterprise settings where maintainability and clarity are critical.

I. Threats to Validity

Although this review followed strict PRISMA guidelines, several validity threats remain. First, the selection was conducted by a single reviewer, which may introduce subjectivity despite well-defined criteria. Second, the search strategy—while broad—may have missed relevant grey literature or emerging preprints not yet indexed in formal databases. Third, the comparative analysis focuses on control-flow modeling and does not extend to performance, resource, or user interaction dimensions, which may also impact practical adoption. These limitations were mitigated through strict screening, quality filters, and reliance on peer-reviewed sources.

IV. Conclusion and Future Work

This seminar paper examines the modeling capabilities and limitations of Petri nets compared to process trees in process mining. The findings indicate a clear tradeoff between expressive power and discoverability: while Petri nets support a broader range of control-flow patterns, including non-free choice and deferred choice, process trees excel in automated discovery due to their block-structured nature and soundness guarantees. These characteristics make Petri nets suitable for advanced modeling scenarios and simulation tasks, whereas process trees are preferred for practical mining and conformance analysis.

Limitations: The study is literature-based and does not include empirical testing on real-life event logs. It also focuses solely on control-flow behavior and does not address performance, resource, or data perspectives of process models.

Evidence Limitations (PRISMA 23b): While the selected 30 studies offer strong insights into the modeling capabilities of Petri nets and process trees, most focus on control-flow constructs within benchmark or synthetic datasets. Only a minority include large-scale real-world case studies, and coverage of advanced patterns (e.g., non-free choice) is uneven across sources.

Review Limitations (PRISMA 23c): The review was conducted by a single researcher without external validation or inter-rater agreement. Despite strict screening and filtering protocols, potential selection bias and subjective judgment during inclusion decisions cannot be entirely ruled out.

Contributions of This Study: This seminar paper provides a structured comparative framework for evaluating Petri nets and process trees using the workflow control-flow pattern lens. It contributes a curated and quality-assessed literature base of 30 peer-reviewed studies, identifies specific tradeoffs in expressiveness and discoverability, and interprets their implications for automated discovery, industrial application, and future hybrid modeling research.

Future Work: Future research should include experimental comparisons of Petri nets and process trees on real-world event logs. This could involve benchmarking discovery accuracy, evaluating model usability in business environments, and exploring hybrid formalisms that combine the strengths of both modeling approaches.

References

[1] S. J. J. Leemans, D. Fahland, and W. M. P. van der Aalst, "Discovering block-structured process models from event logs containing infrequent behaviour," in *Business Process Management Workshops*, Springer, 2014, pp. 66–78.

[2] N. Russell, A. H. M. ter Hofstede, W. M. P. van der Aalst, and N. Mulyar, "Workflow Control-Flow Patterns: A Revised View," BPM Center Report BPM-06-22, 2006.

[3] D. C. de Carvalho, C. C. de Souza Baptista, and F. A. Baia Reis, "Discovering non-free-choice constructs in process mining: a benchmark," in *Information Systems*, vol. 88, 2020.

[4] W. M. P. van der Aalst, *Process Mining: Discovery, Conformance and Enhancement of Business Processes*, Springer, 2011.

[5] W. M. P. van der Aalst, "The application of Petri nets to workflow management," Journal of Circuits, Systems and Computers, vol. 8, no. 1, pp. 21–66, 1998.

[6] F. M. Maggi, A. J. Mooij, and W. M. P. van der Aalst, "An operational decision model for event prediction," Decision Support Systems, vol. 68, 2014.

[7] S. J. J. Leemans, D. Fahland, and W. M. P. van der Aalst, "Discovering block-structured process models from event logs," in Proceedings of the 2013 IEEE Symposium on Computational Intelligence and Data Mining, 2013.

[8] N. Russell, A. H. M. ter Hofstede, W. M. P. van der Aalst, and N. Mulyar, "Workflow Control-Flow Patterns," Technical report BPM-05-01, BPM Center, 2005.

[9] M. J. Page et al., "The PRISMA 2020 statement: An updated guideline for reporting systematic reviews," BMJ, vol. 372, 2021.

[10] D. Fahland, W. M. P. van der Aalst, et al., "Repairing process models to reflect reality," in *Business Process Management*, Springer, 2012, pp. 229–245.

[11] W. M. P. van der Aalst, "Business Process Management Demystified: A Tutorial on Models, Systems and Standards for Workflow Management," in *Lect. Notes Comput. Sci.*, vol. 3098, Springer, 2004, pp. 1–65.

[12] S. J. van Zelst, B. F. van Dongen, and W. M. P. van der Aalst, "Event stream-based process discovery using abstract representations," in *Data-Driven Process Discovery and Analysis*, Springer, 2017, pp. 113–129.

[13] A. J. M. M. Weijters and W. M. P. van der Aalst, "Rediscovering Workflow Models from Event-Based Data Using Little Thumb," in *Integrated Computer-Aided Engineering*, vol. 10, no. 2, pp. 151–162, 2003.

[14] B. F. van Dongen, S. J. J. Leemans, and F. Mannhardt, "Guided process discovery using log skeletons," in *Transactions on Petri Nets and Other Models of Concurrency XII*, Springer, 2017, pp. 1–25.

[15] A. Bolt, S. J. J. Leemans, and W. M. P. van der Aalst, "A framework for filtering event logs based on behavior," in *International Conference on Business Process Management*, Springer, 2019, pp. 40–56.

[16] T. Bayomie, M. Weidlich, and M. Fahland, "A comparative analysis of control-flow modeling constructs in process discovery," in *Information Systems*, vol. 114, 2023, 102128.

[17] M. La Rosa, W. M. P. van der Aalst, and A. H. M. ter Hofstede, "Ensuring model correctness in process discovery," in *Business Process Management Workshops*, Springer, 2009, pp. 161–177.

[18] J. Munoz-Gama, J. Carmona, and M. Sepúlveda, "Conformance checking in the large: Partitioning and topology," in *International Conference on Business Process Management*, Springer, 2013, pp. 130–145.

[19] S. Suriadi, M. Dumas, and A. H. M. ter Hofstede, "Process mining in practice: A case study in a financial organization," in *Information Systems*, vol. 62, 2017, pp. 136–153.

[20] J. Weerdt, M. De Backer, and B. Baesens, "Model-driven process mining: A benchmark study," in *Decision Support Systems*, vol. 54, no. 1, 2013, pp. 601–617.

[21] T. Jouck and B. Depaire, "A comparative exploration of process mining techniques for process discovery from event logs," in *IEEE Transactions on Services Computing*, 2016.

[22] A. Bolt and B. F. van Dongen, "Benchmarking conformance checking techniques with log skeletons and Petri nets," in *International Conference on Business Process Management*, Springer, 2022, pp. 68–83.

[23] E. De Murillas, J. Carmona, and M. Weidlich, "Incremental discovery of process models from event streams," in *Information Systems*, vol. 84, 2019, pp. 214–232.

[24] F. Ongenae, S. Vermeulen, and F. De Turck, "A semantic rule-based method for monitoring clinical pathways," in *Journal of Biomedical Informatics*, vol. 49, 2014, pp. 389–412.

[25] A. Okutan, A. Polyvyanyy, and M. Dumas, "A study of real-life business process event logs," in *International Journal of Cooperative Information Systems*, vol. 29, no. 4, 2020.

[26] B. Kang, M. Weidlich, and A. Solti, "Process discovery using deep learning," in *arXiv preprint arXiv:2204.06636*, 2022.

[27] M. Li, J. Zhu, and D. Fahland, "Evaluating automated process discovery methods with real-life logs," in *Information Systems*, vol. 101, 2021, 101770.

[28] M. Neurauter and M. Weidlich, "Data- and resource-aware process modeling," in *Business & Information Systems Engineering*, vol. 62, 2020, pp. 233–245.

[29] R. Lu and S. Sadiq, "Managing process model complexity via abstract syntax modifications," in *Information Systems*, vol. 32, no. 6, 2007, pp. 877–893.

[30] L. Reinkemeyer, *Process Mining in Action: Principles, Use Cases and Outlook*, Springer, 2020.

[31] A. Berti, M. L. Bernardi, and W. M. P. van der Aalst, "Visual comparison of discovered process models," in *International Conference on Business Process Management*, Springer, 2019, pp. 52–68.